《大模型与智能体技术》实验报告单

姓名：郑茂繁　　学号：26053040　　日期：2026-11-03

一、实验环境记录

硬件环境：搭载 AMDRyzen77800X3D 八核十六线程处理器（4.2GHz，96MBL3 缓存，AM5 平台）、24GBDDR5 内存（单条 24GB，EXPO 已开启、实际运行频率 6000MT/s）与 NVIDIAGeForceRTX5060Ti16GB 显卡（驱动 621.01，CUDA13.5，TDP180W，另含 AMDRadeon 核显），主板为 ASUSB650M-AYWWIFI（BIOS3602）；存储方面 C 盘 96.7GB（剩余 9.6GB）、D 盘 200GB（剩余 167.6GB）、E 盘 200GB（剩余 161.2GB）、F 盘 433.6GB（剩余 188.9GB），显示分辨率 1920×1080；该配置的 16GB 显存可完整容纳 Qwen3-8BQ4_K_M 量化模型（约 4.7GB）及 32K 上下文缓存，实测推理速度约 76tok/s、首字延迟约 150ms，满足本实验中本地大模型部署、对话调用与流式输出的需求，同时在学习的过程中使用 deepseek-flash 进行辅助开发。

软件环境：Windows11 专业版+Python3.12（项目内.venv 虚拟环境）+llama.cppCUDA 预编译推理引擎（llama-server，build11946，CUDA13.4 版）+GGUF 格式的 Qwen3-8B-Q4_K_M 量化模型（约 4.7GB），由 llama-server 对外提供 OpenAI 兼容 HTTP 接口（127.0.0.1:8010），Python 侧仅依赖 openai/requests/numpy 等轻量包负责启动、调用与实验脚本，不依赖 PyTorch、Transformers 等深度学习框架。

开发与依赖：Python 3.12（项目内 .venv）；调用用 openai、requests、numpy、Pillow、matplotlib、scipy；智能体工具 DeepSeek Harness；LoRA 微调用 torch、transformers、peft、trl（CPU 版）；Node.js LTS 用于工具链实验。

二、大模型使用及调用的实践

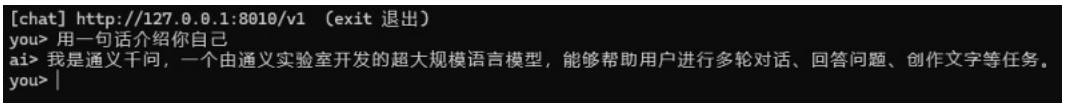
【实验要求（实践一：大模型的使用及调用）】

1. 完成 Ollama 安装，成功拉取并运行 qwen3.5:0.8b 模型；
2. 通过浏览器或 curl 验证本地服务可用；
3. 安装并配置 Chatbox，完成与本地模型的图形化对话；
4. 使用 Python SDK 完成同步与流式两种调用；
5. 运行 chat_loop.py 实现多轮对话，截图记录运行结果。

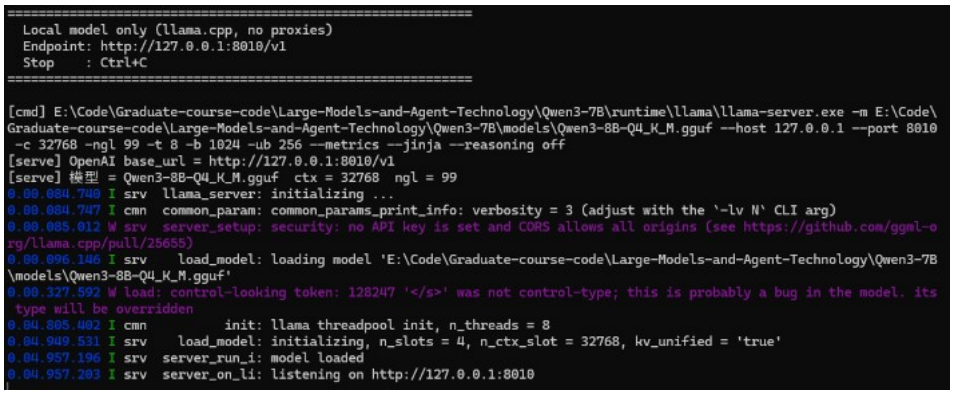
说明：实践一文档中已申明“这里使用的是 llama 而不是 ollama 部署本地模型，所以部分实验步骤不能提供对应证明”，所以第 1 条用 llama.cpp 运行 Qwen3-8B 的 Q4_K_M 量化版本代替。

【实验要求对应的原图（来自实践一文档）】

第 1 条对应原图：

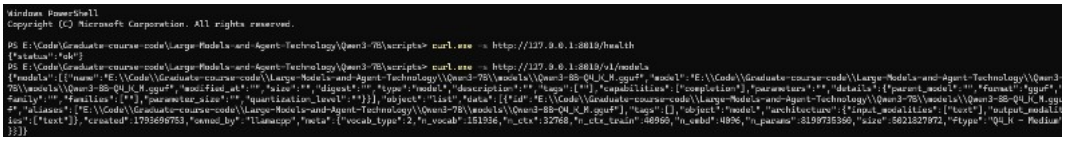


原实践一图：要求 1（Ollama 安装/运行 → 实际用 llama.cpp 加载 Qwen3-8B）



原实践一图：要求 1（Ollama 安装/运行 → 实际用 llama.cpp 加载 Qwen3-8B）

第 2 条对应原图：



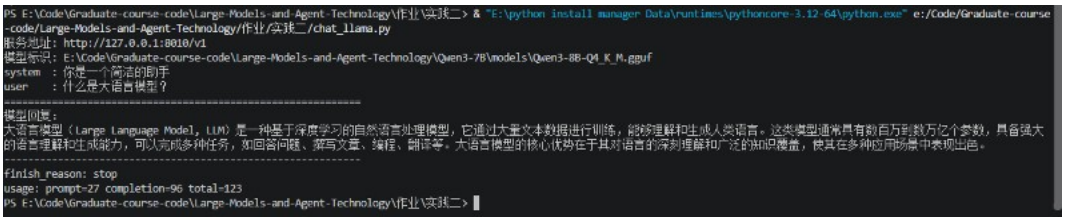
原实践一图：要求 2（curl 验证本地服务）

第 3 条对应原图：



原实践一图：要求 3（Chatbox 图形化对话）

第 4 条对应原图：

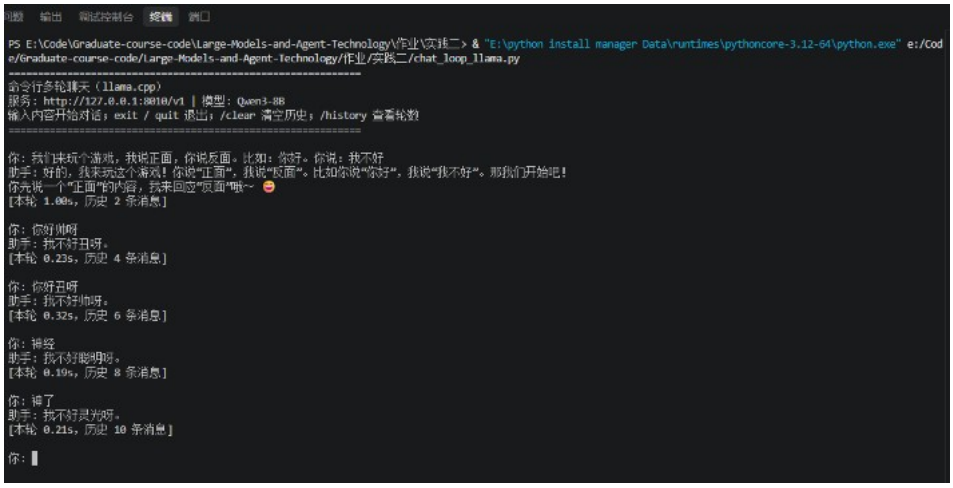


原实践一图：要求 4（Python 同步/流式调用）



原实践一图：要求 4（Python 同步/流式调用）

第 5 条对应原图：



原实践一图：要求 5（chat_loop.py 多轮对话）

【实验过程与结果（原实践一截图）】

环境准备与 Ollama 安装（本机实际用 llama.cpp 代替）：

知识与技能目标

掌握本地大模型服务（Ollama）的安装与使用方法；

掌握利用 Python 程序调用本地大模型的两种基本方式：同步调用与流式调用。

过程与方法目标

具备本地环境搭建与问题排查的能力；

初步建立"部署—调用—验证"的工程化实践流程意识。

原实践一图：环境准备与模型安装

实验内容：大模型的使用及调用

完成 Ollama 的安装与本地大模型部署；

使用命令行与 Python SDK 两种方式与大模型交互。

实验环境

操作系统：Windows / macOS / Linux 均可；

内存：建议 16GB 及以上；

磁盘：预留 10GB 以上空间存放模型文件。

原实践一图：环境准备与模型安装

Ollama 是一款在本地电脑上运行大语言模型（LLM）的开源工具，无需服务器与网络即可使用；

底层基于 llama.cpp 推理框架，以 Docker 风格的命令管理模型；

支持 Llama、Qwen、DeepSeek、Gemma、Mistral 等主流开源模型家族；

安装包内置 Web 服务，默认监听 11434 端口，供程序调用。

官网：https://ollama.com/

原实践一图：环境准备与模型安装

```
# 1. 下载安装包（Windows）
#    https://ollama.com/download/OllamaSetup.exe
#    双击安装，完成后自动注册系统服务

# 2. 验证安装（任意终端）
ollama --version

# 3. 拉取本实验用模型（约 1.0GB，256K 上下文）
ollama pull qwen3.5:0.8b

# 4. 命令行直接对话验证
ollama run qwen3.5:0.8b "用一句话介绍你自己"
```

安装要点

Windows 版安装后 Ollama 常驻后台，无需手动启动服务；

常用命令

ollama list 查看本地模型；

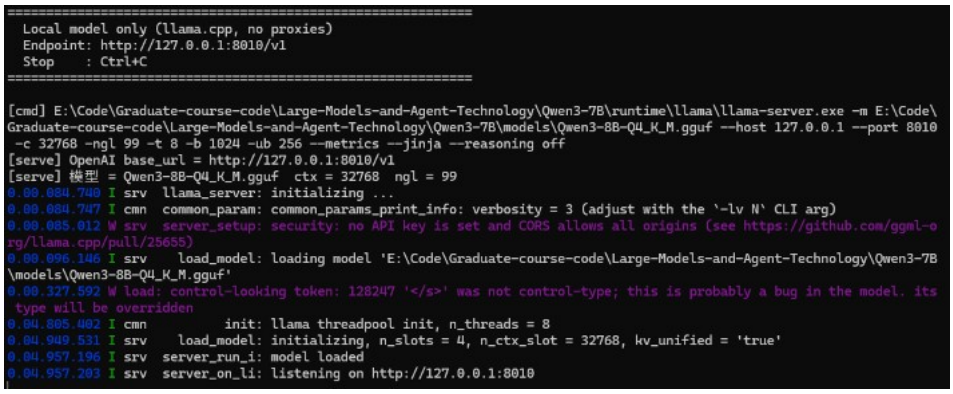
ollama rm <模型名> 删除模型；

ollama serve 手动启动服务（默认端口 11434）。

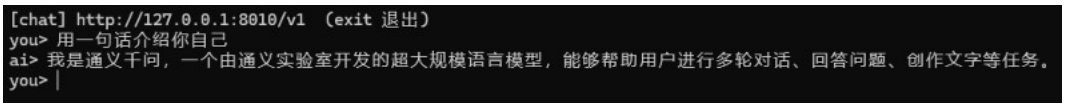
原实践一图：环境准备与模型安装

```
PS E:\Code\Graduate-course-code\Large-Models-and-Agent-Technology\Qwen3-7B> .\runtime\llama\llama-server.exe --version
version: 0.7.0-dev (build 11946, commit 8ca4060b7)
built with Clang 20.1.8 for Windows x86_64
```

原实践一图：环境准备与模型安装



原实践一图：环境准备与模型安装



原实践一图：环境准备与模型安装

服务验证：

打开浏览器访问 http://localhost:11434，显示 "Ollama is running" 即服务正常；

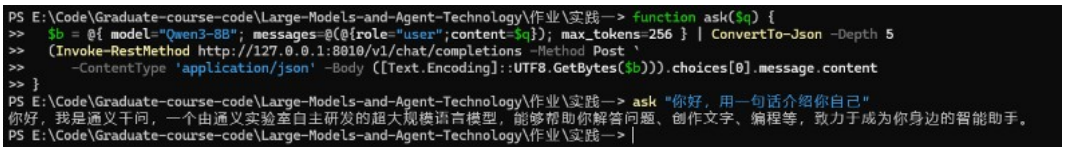
REST 接口测试（PowerShell / bash）

curl http://localhost:11434/api/generate -d '{"model": "qwen3.5:0.8b", "prompt": "你好", "stream": false}'

接口返回 JSON，其中 response 字段为模型输出；

该 REST 接口即 Python SDK 的底层通道，SDK 本质是对 HTTP 接口的封装。

原实践一图：服务验证



原实践一图：服务验证

Chatbox 图形界面：

Chatbox 简介

Chatbox 是一款开源的跨平台 AI 桌面客户端（Windows / macOS / Linux）；

无需编写代码即可与本地 Ollama 模型对话，适合日常使用与提示词调试。

配置步骤

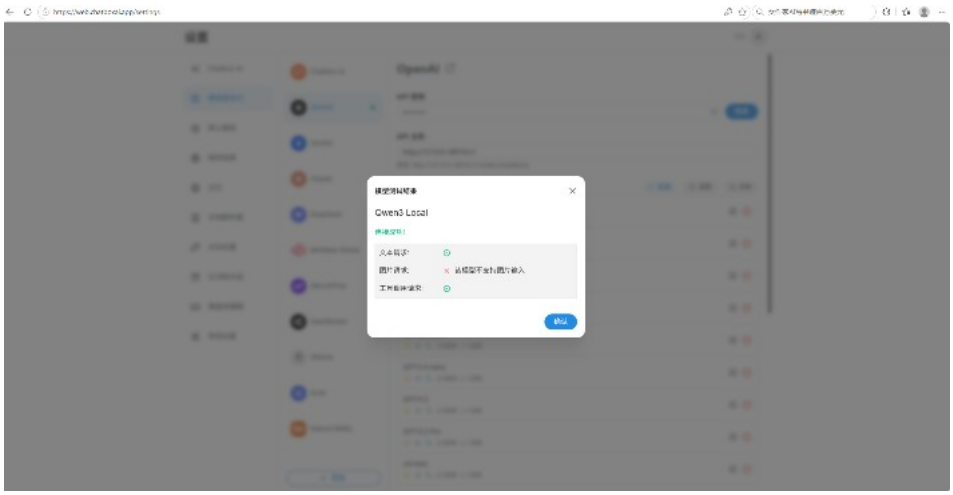
1，下载安装：https://chatboxai.app/（社区版免费）；

2，打开"设置" → "模型提供方" → 添加 "Ollama"；

3，API 域名填 http://127.0.0.1:11434，点击刷新获取本地模型列表；

4，选择 qwen3.5:0.8b，保存后即可开始图形化对话。

原实践一图：Chatbox 对话



原实践一图：Chatbox 对话



原实践一图：Chatbox 对话

Python 调用准备、同步调用、流式调用：

```
# 创建一个名为 ollama-env 的 Conda 环境，Python 版本可自己改
conda create -n ollama-env python=3.13 -y

# 激活环境：Windows / macOS / Linux 都一样
conda activate ollama-env

# 安装 Ollama Python SDK
python -m pip install ollama

# 验证
python -c "import ollama; print('ollama import ok')"

# 退出环境
conda deactivate
```

说明

SDK 通过 HTTP 访问本地 11434 端口，调用前确保 Ollama 服务已启动；

SDK 与 Ollama 版本相互独立，可单独升级；

完整可运行代码见《实践代码/实践一/》。

原实践一图：Python 同步/流式调用

```
from ollama import chat

response = chat(
    model="qwen3.5:0.8b",
    messages=[
        {"role": "system", "content": "你是一个简洁的助手"},
        {"role": "user", "content": "什么是大语言模型？"},
    ],
)
print(response.message.content)
```

代码要点

messages 采用角色—内容结构：syste / user / assistant；

调用返回完整结果，适合短问答场景；

response.message.content 为模型回 文本。

原实践一图：Python 同步/流式调用



原实践一图：Python 同步/流式调用

```
from ollama import chat

stream = chat(
    model="qwen3.5:0.8b",
    messages=[{"role": "user", "content": "写一首关于秋天的短诗"}],
    stream=True,
)
for chunk in stream:
    print(chunk["message"]["content"], end="", flush=True)
```

同步 vs 流式

同步：一次性返回全文，代码简单，等待时间长；

流式：逐字输出，首字延迟低，交互体验好；

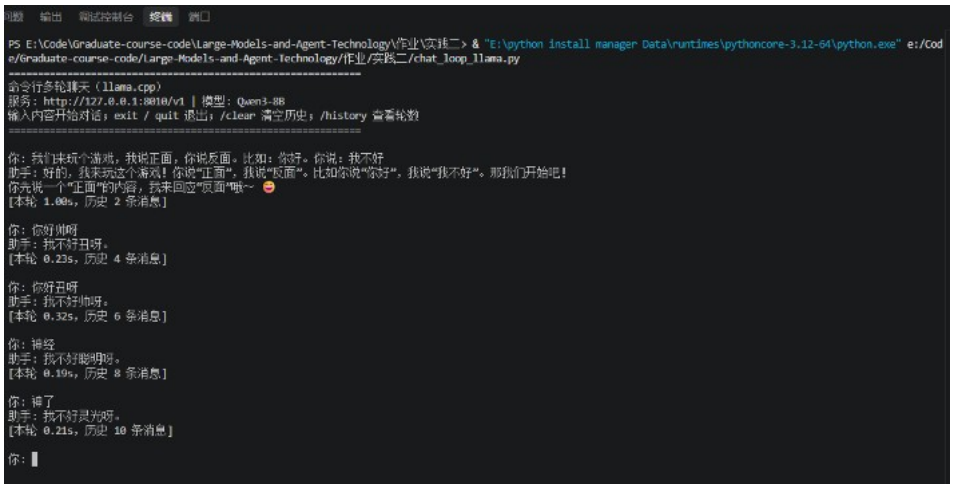
大模型应用（聊天界面）普遍采用流式输出。

原实践一图：Python 同步/流式调用



原实践一图：Python 同步/流式调用

命令行多轮聊天：



原实践一图：命令行聊天

三、大模型智能体工具的实践

【实验要求（实践二：大模型智能体工具的安装和使用）】

1. 完成 Node.js（LTS 24+）安装与 npm 国内源配置，安装 DeepSeek Harness（dsh）并完成 Provider 与 Workspace 配置；

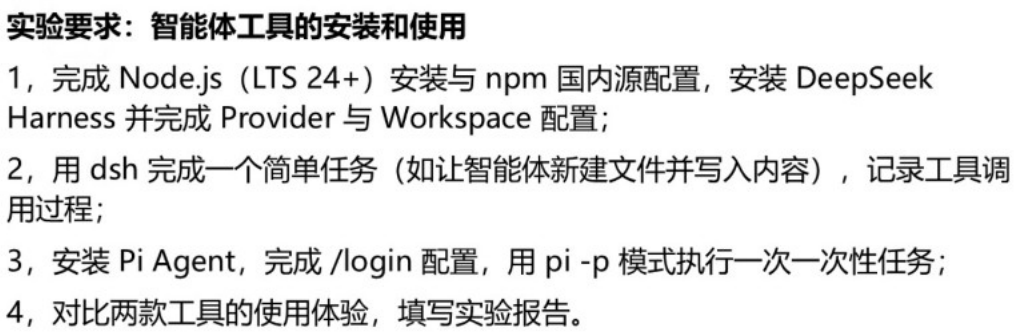
2. 用 dsh 完成一个简单任务（如让智能体新建文件并写入内容），记录工具调用过程；

3. 安装 Pi Agent，完成 /login 配置，用 pi -p 模式执行一次一次性任务；

4. 对比两款工具的使用体验，填写实验报告。

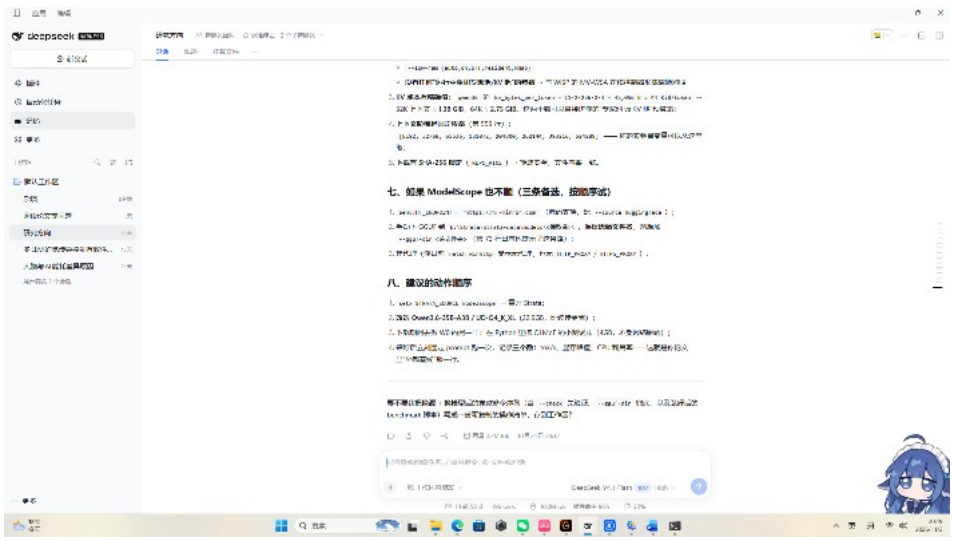
【实验要求对应的原图（来自实践二文档）】

实验要求说明页：



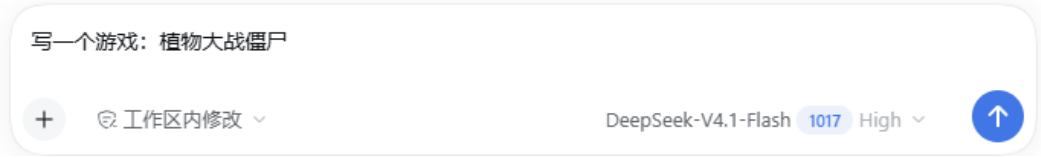
原实践二图：实验要求说明

第 1 条对应原图（Node.js 安装与 npm 国内源配置）：



原实践二图：要求 1（Node.js / npm 配置）

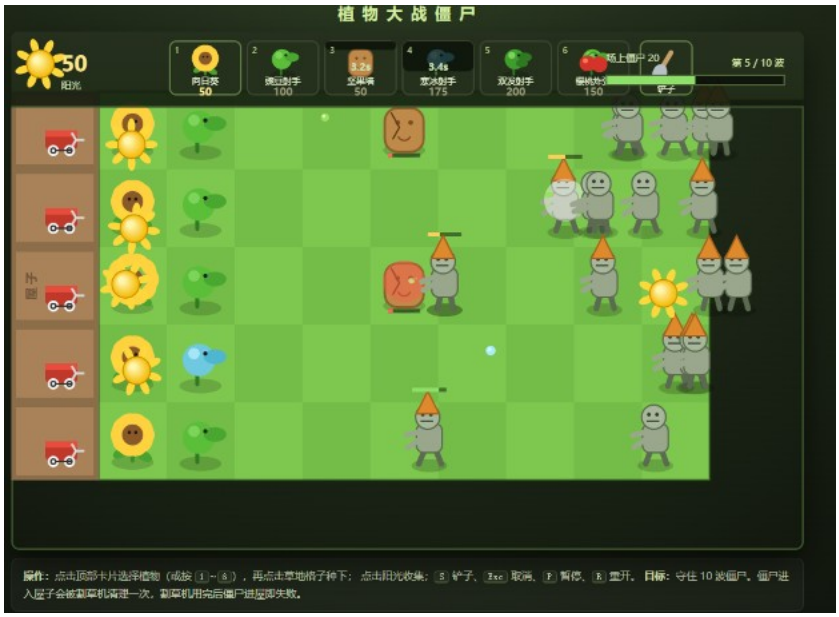
第 2 条对应原图（用 dsh 做简单任务并记录工具调用）：



原实践二图：要求 2（dsh 完成简单任务）

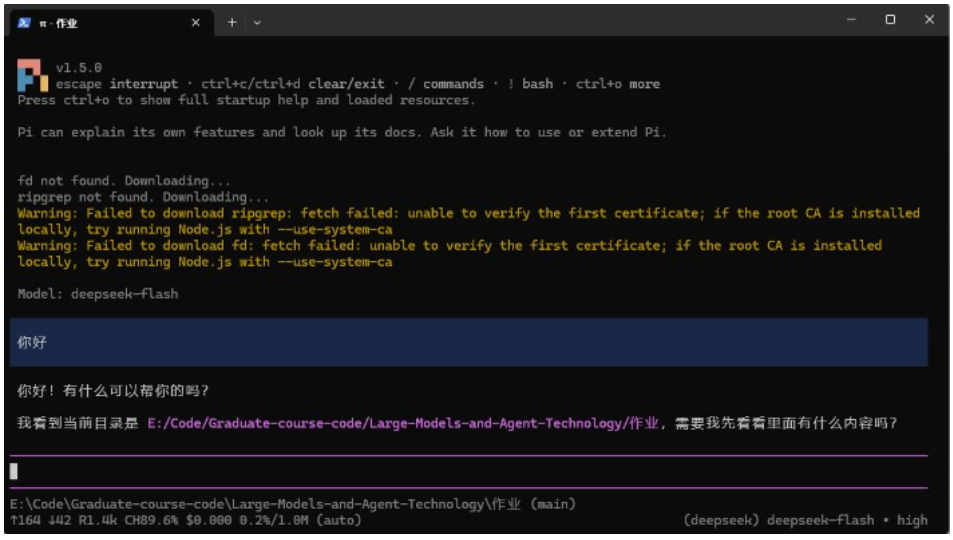


原实践二图：要求 2（dsh 完成简单任务）

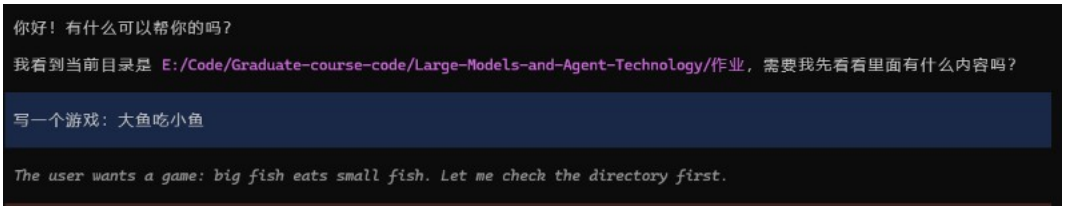


原实践二图：要求 2（dsh 完成简单任务）

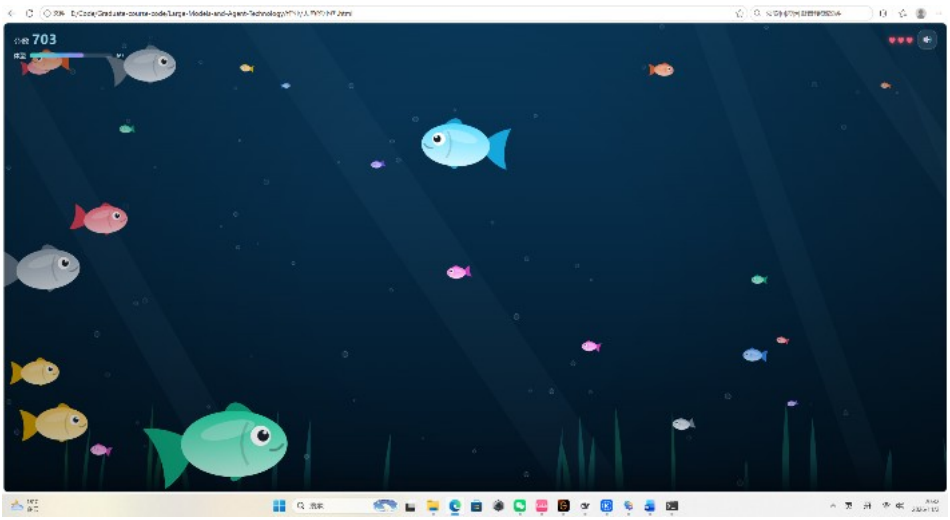
第 3、4 条对应原图（Pi Agent 与工具对比）：



原实践二图：要求 3/4（Pi Agent 与对比）



原实践二图：要求 3/4（Pi Agent 与对比）



原实践二图：要求 3/4（Pi Agent 与对比）

【实验过程与结果（原实践二截图）】

Node.js 环境安装：

```
# 1. 下载安装 Node.js (Windows)
#    官网: https://nodejs.org/ → 下载 LTS 版 .msi
#    国内镜像: https://npmmirror.com/mirrors/node/
#    双击安装，自动配置环境变量

# 2. 新开 PowerShell 验证安装
node -v    # v24.x (LTS)
npm -v

# 3. npm 换国内源 (npmmirror 淘宝镜像)
npm config set registry https://registry.npmmirror.com

# 4. 验证换源成功 (输出镜像地址即生效)
npm config get registry

# 5. 还原官方源 (如需发布自己的包)
npm config set registry https://registry.npmjs.org
```

原实践二图：Node.js 环境安装

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\zmf20\Desktop> node -v
v26.13.0
PS C:\Users\zmf20\Desktop>
```

原实践二图：Node.js 环境安装

安装 DeepSeek Harness：

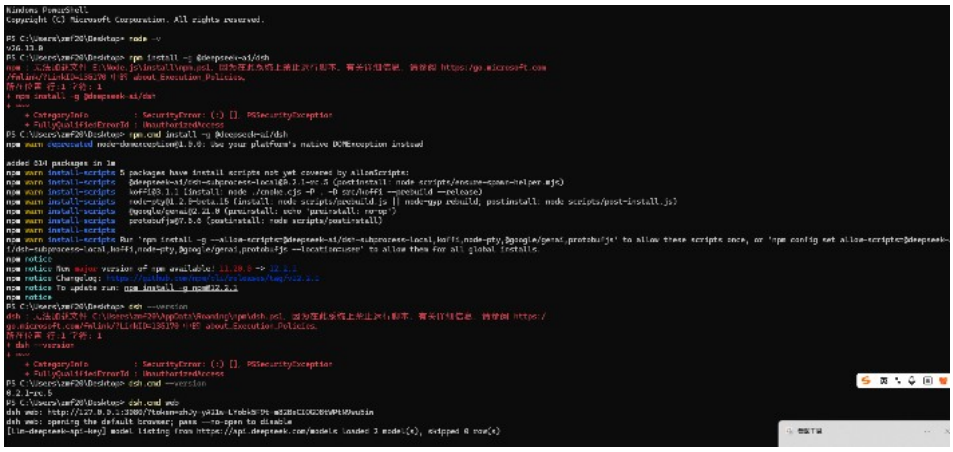
```
# 环境要求：Node.js 24+
node -v

# 全局安装 dsh CLI
npm install -g @deepseek-ai/dsh

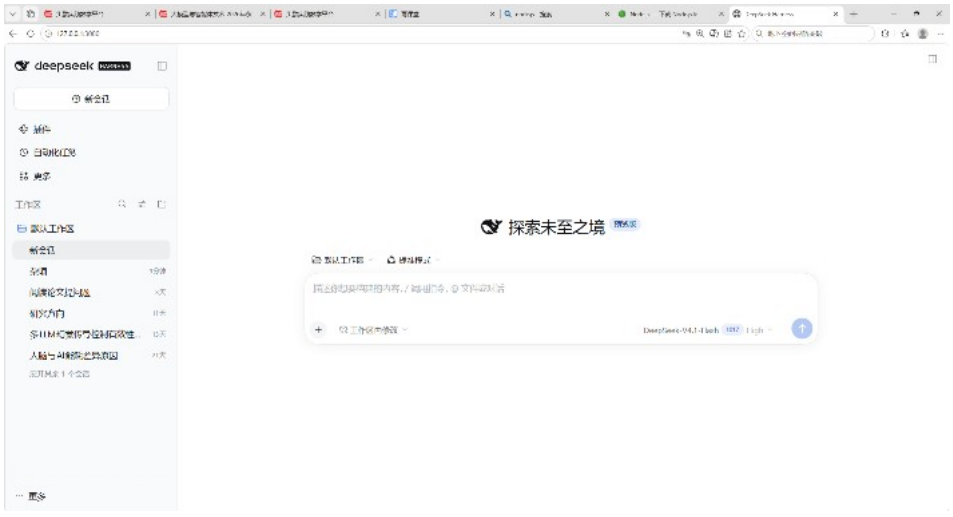
# 验证安装
dsh --version

# 启动（首次启动进入 Web UI 引导）
dsh web
```

原实践二图：安装 dsh



原实践二图：安装 dsh



原实践二图：安装 dsh

首次配置流程：

首次启动 dsh 后按引导完成三步配置：

1，配置 Provider：选择模型提供方（DeepSeek API Key 或其他兼容接口）；

2，配置 Workspace：指定智能体可访问的工作目录；

3，按需引入社区插件：文件读写、Shell 执行、浏览器等技能插件。

配置完成后即可在会话中用自然语言下达任务，观察模型的工具调用过程。

原实践二图：首次配置流程

四种运行模式：

Standard：

通用对话与任务执行模式，适合日常使用；

Code：

代码智能体模式，面向编程场景优化；

Minimal：

最小化模式，仅加载核心能力，启动快；

Creator：

创建者模式，用于组装自定义智能体；

每次运行全程可追踪（Every Run is Traceable），便于复盘。

原实践二图：四种运行模式

安装 Pi Agent：

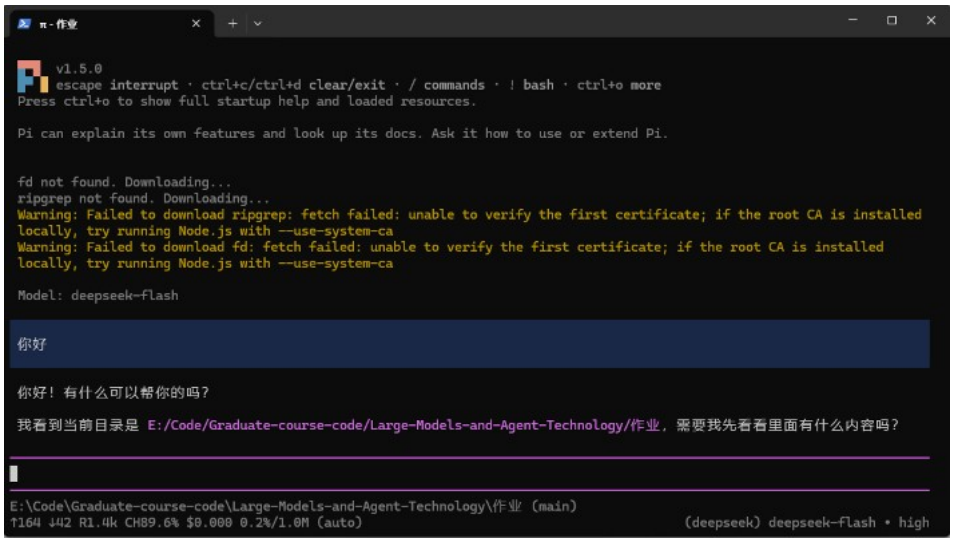
```
# 环境要求：Node.js 22.19+
node -v

# npm 全局安装（推荐 --ignore-scripts）
npm install -g --ignore-scripts @earendil-works/pi-coding-agent

# Windows 也可用官方脚本
# powershell -c "irm https://pi.dev/install.ps1 | iex"

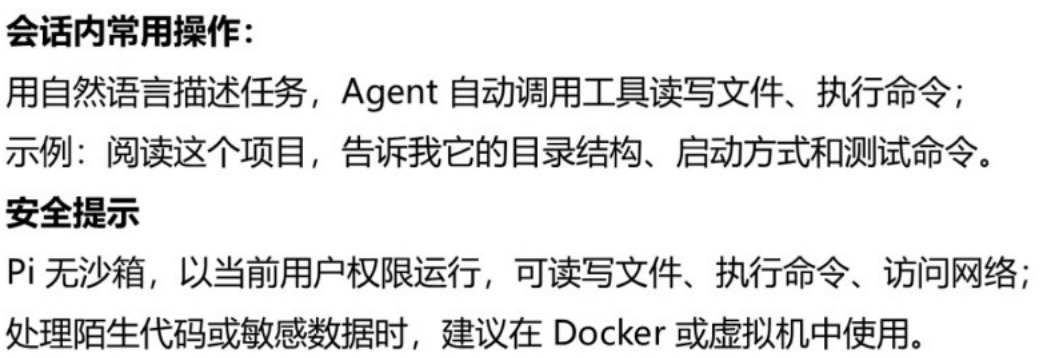
# 进入项目目录启动
cd /path/to/project
pi
```

原实践二图：安装 Pi Agent



原实践二图：安装 Pi Agent

Pi Agent 基本用法：



原实践二图：Pi Agent 基本用法

四、大模型的微调实践

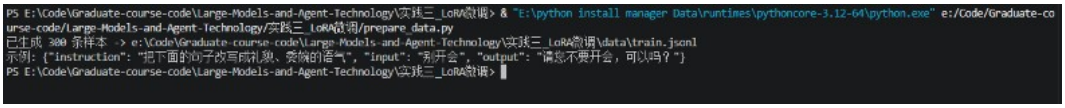
【实验要求（实践三：大模型的 LoRA 微调）】

1. 理解 LoRA（Low-Rank Adaptation）参数高效微调的原理；
2. 完成对 1B 以下小模型 Qwen2.5-0.5B-Instruct 的 LoRA 微调全流程（准备数据、训练、对比）；
3. 对比微调前后的输出效果；
4. （选做）合并 LoRA 权重并转换为 GGUF，导入 llama.cpp 使用。

【实验过程与结果（原实践三截图）】

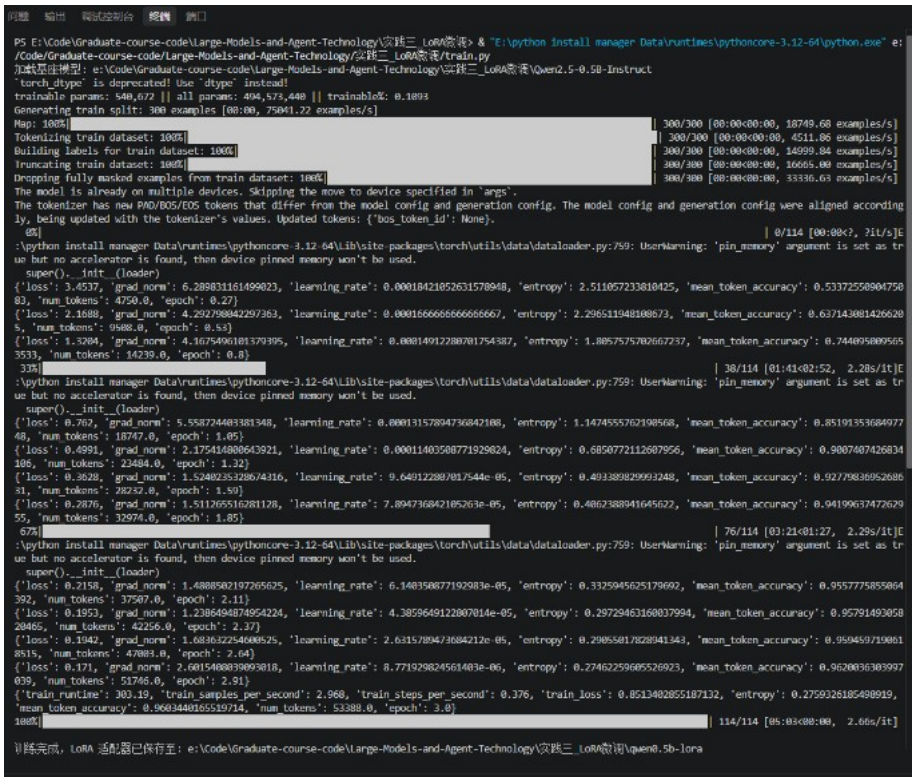
本次微调的是 Qwen2.5-0.5B-Instruct（约 5 亿参数），任务是把“把门关上”这类命令句改成“麻烦您把门关上，可以吗？”这种礼貌说法。用 LoRA 微调：r=8、alpha=16、dropout=0.05，只改 q_proj、v_proj 两层，可训练参数约 54 万（占 0.1%）。

数据：用 prepare_data.py 生成 300 条“命令句—礼貌句”配对数据。



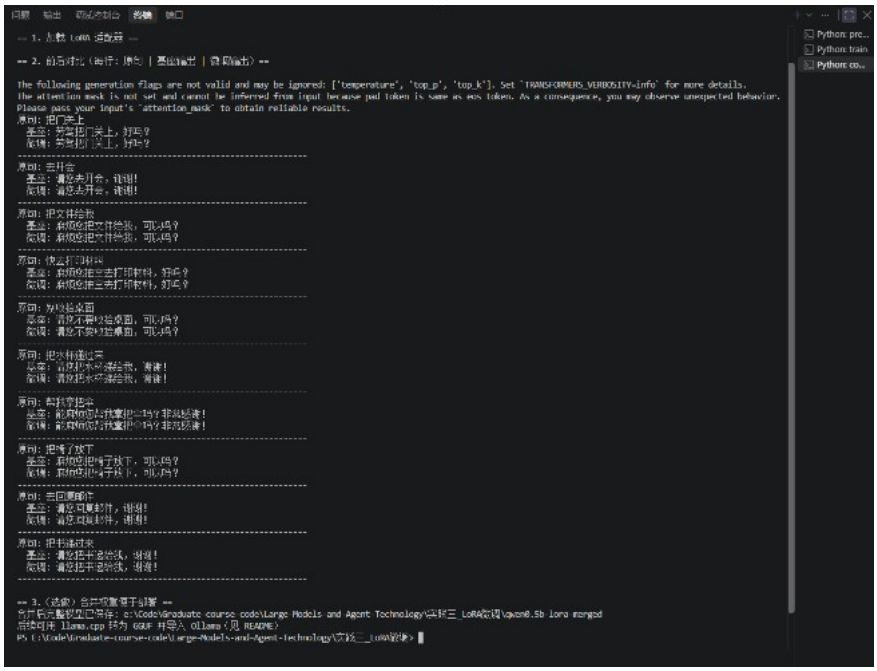
原实践三图：生成训练数据（300 条）

训练：3 轮共 114 步，学习率约 2e-4；本机 torch 为 CPU 版，训练约 5 分钟，损失从 3.45 降到 0.85。



原实践三图：LoRA 训练日志

对比：compare.py 用 10 句话分别问基座模型和微调后的模型。基座已能改写，但输出有时重复、格式不稳；微调后句子更统一。



原实践三图：微调前后输出对比

结果：qwen0.5b-lora 为 LoRA 参数（约 2 MB），qwen0.5b-lora-merged 为合并后的完整模型；选做部分也做了合并。

五、收获与建议

三个实践做下来，感觉大模型这块“跑起来”和“用起来”之间还有不少活。部署时要先确认显卡驱动、CUDA、模型格式（GGUF）、端口这些，任何一步不对都跑不起来；能跑之后还要调参数、写调用代码、处理报错，才能稳定使用。

按实验要求对照：实践一 1～5 条都完成（第 1 条按文档说明用 llama.cpp 代替 Ollama）；实践二第 1、2、4 条完成，第 3 条（Pi Agent）因没有使用而未完成；实践三 1～3 条完成，第 4 条选做也做了合并。

收获：弄懂了本地模型怎么被加载和调用（llama.cpp + OpenAI 接口）；学会用 Python 调大模型（同步、流式、多轮）；第一次用智能体工具，感觉它能帮我干活，但不能完全相信；学会 LoRA 微调，知道大模型可以通过“小补丁”改变行为。

建议：希望课程多给一些现成脚本和环境说明，减少大家踩环境的坑；实验报告如果能把每一步的截图要求写清楚，整理起来会更容易。